Plano de 8 semanas: profiling de workloads agênticos

Preparação para candidatura de PhD com o Prof. Yiwei Zhao (Indiana University Bloomington)

Objetivo final: um repositório no GitHub + um relatório de 4 páginas com números reais, para anexar ao email e à candidatura. Prazo da candidatura: 15 de dezembro de 2026, então o plano termina em torno de **10 de dezembro**, com folga.

Semana 1 (19-25 out): Setup e agente base

- Escolha o stack: Python + um framework leve (ou escreva o loop ReAct à mão, que é melhor para instrumentar).
- Monte o agente com 3 ferramentas: busca (Wikipedia ou similar), calculadora e execução de código em sandbox.
- Rode com Ollama/llama.cpp local ou com uma API, só para validar que funciona.
- **Para vLLM/SGLang você precisa de GPU**: alugue uma (RunPod, Vast.ai, Lambda ou Colab Pro). Uma GPU de 24 GB basta para modelos de 7-8B.
- **Entregável:** agente funcionando em 5 tarefas de teste.

Semana 2 (26 out-1 nov): Instrumentação e benchmark

- Registe por passo: timestamp, tempo de LLM (TTFT e geração), tempo de ferramenta, tokens de entrada/saída, tokens reaproveitáveis (prefixo repetido).
- Guarde tudo em traces JSON, um por tarefa.
- Escolha 2 workloads: por exemplo, **HotpotQA** (multi-hop com busca) e **GSM8K ou um subconjunto do SWE-bench Lite** (código).
- **Entregável:** pipeline que corre 100 tarefas e gera os traces.

Semana 3 (2-8 nov): Caracterização

- Corra o benchmark completo e analise:
  - Onde o tempo vai (LLM vs. ferramentas vs. overhead).
  - Como o contexto cresce por passo.
  - Que fração dos tokens de entrada é prefixo repetido.
  - Distribuição do número de passos e da latência (P50/P95).
- Faça gráficos limpos (matplotlib): breakdown de tempo, crescimento de contexto, CDF de latência.
- **Entregável:** 3-4 gráficos e a conclusão principal ("X% do prompt é repetido entre passos", por exemplo).

Semana 4 (9-15 nov): Servir com vLLM e prefix caching

- Suba o modelo no vLLM com prefix caching **ligado e desligado**.
- Compare TTFT, latência por tarefa e throughput.
- Repita com SGLang (RadixAttention), se houver tempo.
- **Entregável:** tabela comparando as configurações. Este é o primeiro resultado "de sistemas" do projeto.

Semana 5 (16-22 nov): Carga concorrente e otimização

- Simule 8/16/32 agentes em paralelo e meça como latência e throughput escalam.
- Teste uma otimização simples, por exemplo:
  - sobrepor chamadas de ferramenta com geração (execução assíncrona);
  - ordenar/agendar requisições para maximizar reuso de cache;
  - reduzir o contexto (truncamento ou sumarização) e medir o custo em acurácia.
- **Entregável:** uma otimização com ganho medido e o trade-off documentado.

Semana 6 (23-29 nov): Relatório e repositório

- Escreva o relatório de ~4 páginas: motivação, setup, caracterização, resultados, limitações, trabalho futuro.
- Polir o GitHub: README com resumo dos resultados, gráficos, instruções para reproduzir (um comando ou script).
- **Página de resultados (React + Tailwind CSS):** use o index.html do projeto, troque o objeto DATA pelos seus números reais e publique no GitHub Pages (Settings > Pages > branch main, pasta /docs). Linke a página no email e no CV.
- **Entregável:** repositório público + PDF do relatório + página de resultados online.

Semana 7 (30 nov-6 dez): Contato e candidatura

- **Envie o email ao Prof. Zhao** (ele pediu para escrever agora). Inclua: CV, histórico, 3-4 frases de interesses de pesquisa, link do repositório e 2 linhas com o resultado principal.
- Preencha as candidaturas em **Intelligent Systems Engineering e Computer Science**, como ele sugere, mencionando o nome dele.
- Prepare o statement of purpose referenciando o projeto.

Semana 8 (7-13 dez): Folga

- Corrija erros, peça feedback de um professor ou colega e **submeta até 10 de dezembro**, não no último dia.

Dicas

- **Não espere a semana 7 para escrever ao professor.** Se quiser, mande um email curto na semana 4 com os primeiros resultados; mostra iniciativa e dá tempo de ele responder.
- Registe tudo desde o início (seeds, versões, configs) para o projeto ser reproduzível.
- Resultados negativos também valem, desde que bem analisados ("prefix caching ajudou só 5% porque...").
- Exporte os resultados de cada experimento para JSON desde cedo; isso facilita preencher a página de resultados na semana 6.
- Se a semana 5 apertar, corte SGLang e a segunda otimização, mas mantenha a caracterização e a comparação do prefix caching.